챕터 4. 비동기 MSA - Kafka로 서비스를 분리하다

동기 호출로 서비스를 운영한 지 며칠 뒤, 오픈이는 에러 로그가 갑자기 늘어나는 것을 확인했습니다. 상품 서비스가 죽은 것이 원인이었습니다. 이후 쿠버네티스가 상품 서비스를 복구하자 에러 로그도 멈췄습니다.

문제는 그 사이에 들어온 주문이 전부 실패한 것입니다. 주문 서비스가 상품 서비스를 호출했지만, 상품 서비스가 죽어 있어 주문 실패가 발생했습니다.

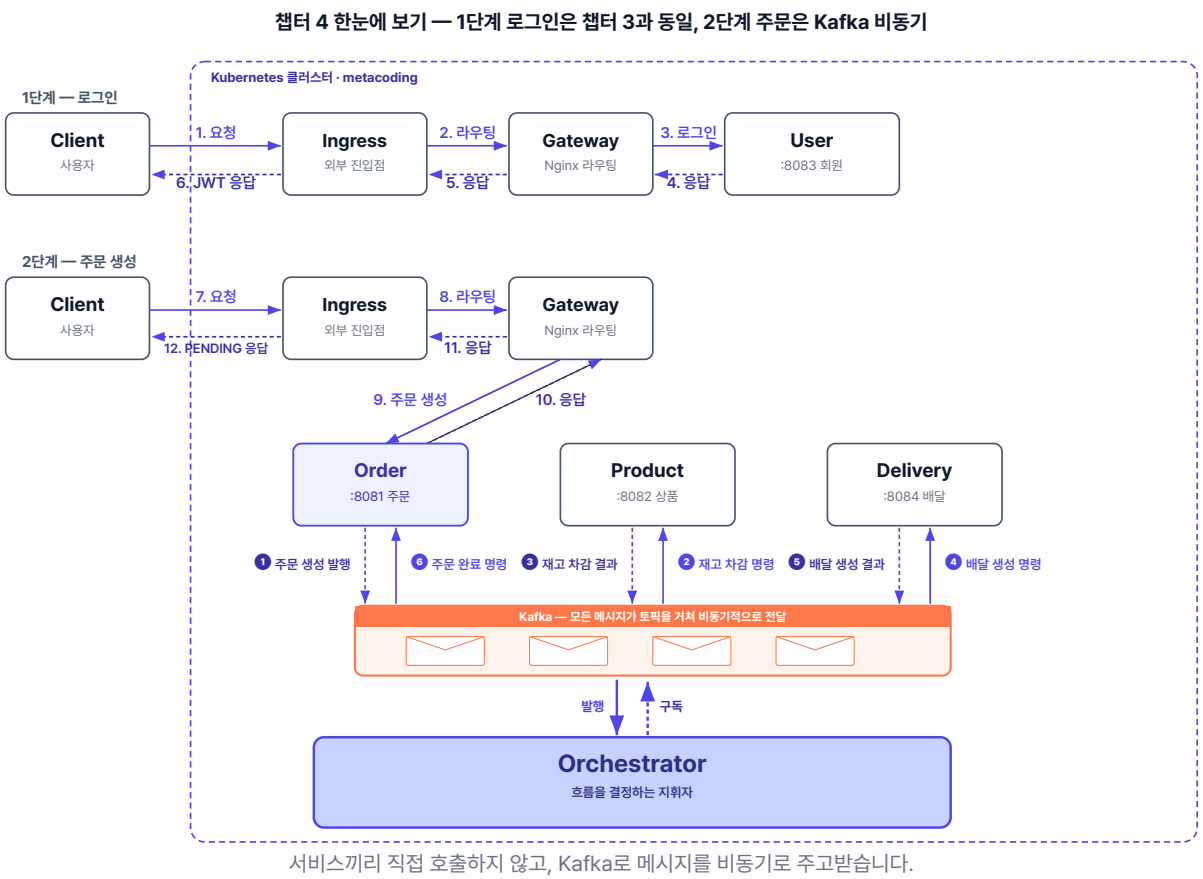
서로 직접 호출하니까, 하나가 죽으면 호출한 쪽도 같이 실패하는구나.

상황을 확인한 선배가 다가왔습니다.

선배 : "이거 지난번에 얘기했던 것처럼, 서비스끼리 동기적으로 직접 호출해서 그래요. 호출한 쪽은 상대가 응답할 때까지 기다리는데, 그 상대가 죽으면 결국 요청이 실패하는 거죠. 그래서 MSA는 동기 방식보다 메시지를 주고받는 비동기 방식을 써야 해요."

오픈이 : "메시지요? 메시지 방식은 어떻게 다른가요?"

선배 : "보내는 쪽은 메시지를 발행하고, 응답을 기다리지 않고 다음 작업을 진행해요. 받는 쪽은 자기 차례에 그 메시지를 읽고 다시 메시지를 발행하죠. 메시지는 한 번 발행되면 바로 사라지지 않아서, 받는 쪽이 잠깐 죽어도 메시지는 그대로 남아 있다가 복구되면 그때 처리돼요."



서비스끼리 직접 호출하지 않고, Kafka로 메시지를 비동기로 주고받습니다.

그림 4-1. 챕터 4 한눈에 보기 - 서비스·Orchestrator·Kafka 3층 구조

준비하기

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/start.git
cd start/ex03
```

완성 코드는 final 레포(`https://github.com/metacoding-12-msa/final`)의 `ex03` 폴더에서 확인할 수 있습니다.

2. 파일 구조

ex03 디렉토리

```
ex03/
├── db/                 # MySQL
├── delivery/           # 포트 8084
├── gateway/            # Nginx API Gateway
├── k8s/                # Kubernetes YAML 파일 (kafka 포함)
├── orchestrator/       # Kafka 워크플로우 조율 (이번 챕터 신규)
├── order/              # 포트 8081
├── product/            # 포트 8082
└── user/               # 포트 8083
```

서비스마다 패키지 구조가 조금씩 다르므로, 코드를 작성할 파일 경로는 각 실습 코드블록 바로 위에서 안내합니다.

3. Kafka

이 챕터는 Kafka를 별도로 설치하지 않습니다. Kubernetes YAML(`k8s/kafka/`)로 Minikube 안에 띄우므로, 챕터 3에서 준비한 Docker Desktop과 Minikube만 있으면 됩니다.

4. 실습 순서

1. order-service의 REST 호출을 Kafka 이벤트 발행으로 교체
2. product-service · delivery-service에 Kafka Consumer/Producer 추가
3. 새 서비스 **orchestrator**에서 워크플로우 조율 로직 작성
4. Kubernetes에 Kafka + orchestrator 배포
5. 정상 주문 / 품절 보상 시나리오 검증

4.1 동기 호출의 한계 - 한 서비스가 멈추면 전체가 멈춘다

동기적 방식과 비동기적 방식을 비유를 통해 알아보겠습니다.

4.1.1 카운터 대기 vs 진동벨

먼저 커피를 주문하면 카운터에서 대기하는 방식입니다. 내가 주문한 커피가 나와야 다음 손님이 주문할 수 있습니다. 만약 커피 머신이 고장 나면 뒤에 줄 선 손님 전부가 기다려야 합니다. 이렇게 요청을 보낸 쪽이 응답이 올 때까지 기다리는 방식이 **동기(synchronous)** 호출입니다.

반대로 커피 주문 후 진동벨을 받으면 자리에 앉아 다른 일을 할 수 있습니다. 커피가 완성되면 벨이 울립니다. 뒤에 줄 선 손님도 곧바로 주문할 수 있습니다. 커피 머신이 멈춰도 주문을 먼저 받아 두고, 고친 뒤에 처리할 수 있습니다. 이렇게 요청을 보낸 쪽이 기다리지 않고 결과가 준비되면 따로 받는 방식이 **비동기(asynchronous)** 호출입니다.

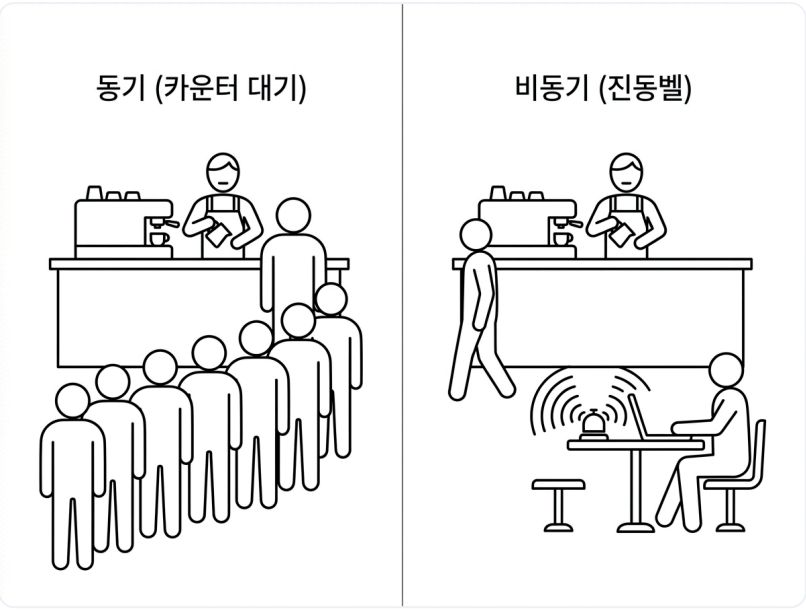


그림 4-2. 동기 vs 비동기 통신

카페의 진동벨처럼, MSA의 서비스도 비동기 통신을 위해 메시지를 사용합니다. 그렇다면 이 메시지는 어떤 방식으로 주고받을까요?

4.2 Kafka - 메시지를 전달하는 우체국

4.2.1 프로듀서·토픽·컨슈머

MSA 구조에서 비동기 메시지는 **Kafka**를 통해 주고받습니다. Kafka는 시스템 사이에서 비동기 통신을 담당하며, 메시지를 안전하게 전달해 주는 역할을 합니다.

> **Apache Kafka**는 대량의 메시지를 빠르고 안정적으로 주고받기 위해 만들어진 **분산 메시지 시스템**입니다. 받은 메시지를 바로 지우지 않고 일정 기간 보관하기 때문에, 필요하면 **지난 메시지를 다시 꺼내 볼 수도** 있습니다. **높은 처리량과 확장성** 덕분에 대규모 서비스에서 널리 사용합니다.

Kafka는 우체국과 같은 역할을 합니다. 발신자가 우편을 부치면, 우체국은 **일반 편지, 등기, 특송, 택배처럼 종류별로 나눠** 따로 보관합니다. 그리고 **집배원은 자기 담당의 칸에서만 우편을 꺼내 갑니다**. 우체국을 사이에 두고 우편을 주고받기 때문에, 비동기적으로 각자의 시간에 일을 처리할 수 있습니다.

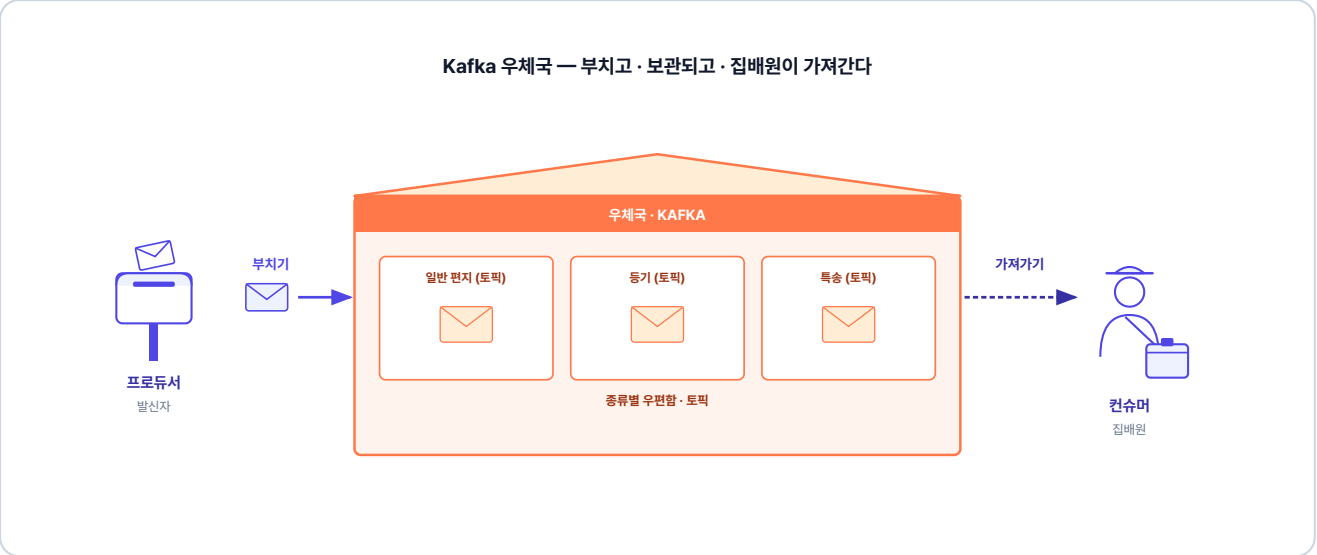


그림 4-3. Kafka 우체국 - 메시지를 종류별로 보관하고 전달하는 구조

그림 4-3의 Kafka 요소의 역할을 정리하면 다음과 같습니다.

구성요소	우체국 비유	하는 일
Kafka	우체국	메시지를 받아 보관하고 전달
토픽(Topic)	종류별 우편함 (일반/등기/특송 등)	메시지를 목적별로 나눠 담음
프로듀서(Producer)	발신자	토픽에 메시지를 발행
컨슈머(Consumer)	집배원	토픽을 구독해 메시지를 처리

프로듀서가 토픽에 메시지를 보내는 것을 **발행**(Publish), 컨슈머가 특정 토픽을 지정해 두고 메시지를 가져와 처리하는 것을 **구독**(Subscribe)이라고 합니다.

우체국이 우편을 일반·등기·특송으로 나눠 담듯, Kafka도 메시지를 **토픽이라는 우편함에 종류별로 나눠 담습니다. 토픽마다 이름이 있어서**, 프로듀서는 보낼 메시지에 맞는 **토픽 이름으로 발행**하고 컨슈머는 자기가 맡은 **토픽 이름으로 구독**합니다.



그림 4-4. Kafka의 프로듀서·토픽·컨슈머 구조

그렇다면 같은 일을 하는 컨슈머가 여러 대 떠 있을 때는 메시지를 어떻게 나눠 받아야 할까요?

4.2.2 컨슈머 그룹

상품 서비스를 2대로 늘려 운영한다고 가정해 보겠습니다. 두 서버가 같은 토픽을 구독하면 "재고 1개 차감" 메시지를 둘 다 받아 처리해, 재고가 2개 줄어듭니다. 이 중복 처리를 막아 주는 것이 **컨슈머 그룹**입니다. **같은 일을 하는 서버를 한 그룹으로 묶으면, 그 메시지는 그룹 안의 한 서버에만 전달됩니다.**

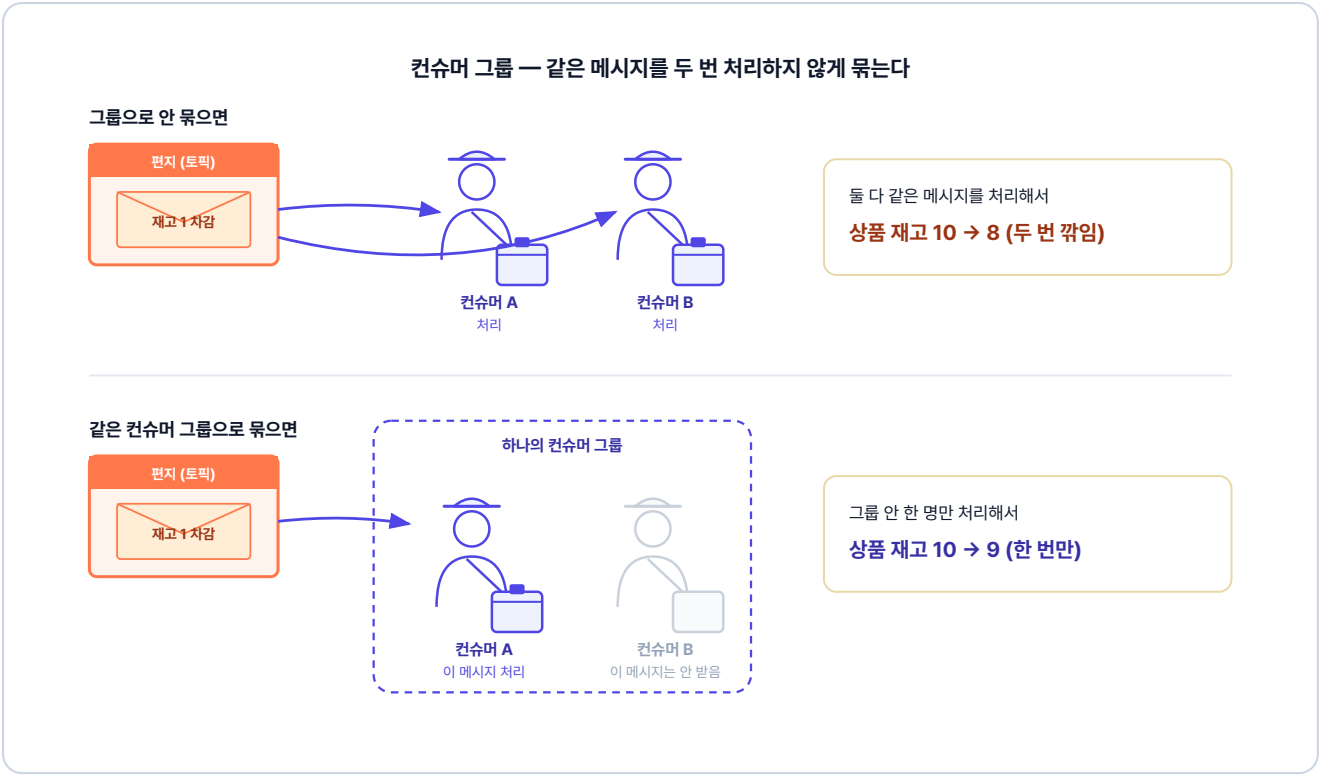


그림 4-5. 컨슈머 그룹 - 묶으면 한 번만 처리

Kafka 더 알아두기

- **컨슈머가 읽어도 메시지는 사라지지 않는다**: Kafka는 전통적인 메시지 큐와 달리, 컨슈머가 메시지를 읽어가도 파일 시스템에 그대로 보관합니다(기본 설정은 7일). 덕분에 서로 다른 컨슈머 그룹이 동일한 메시지를 각자의 속도대로 처음부터 다시 읽을 수 있고, 장애가 발생했을 때도 원하는 시점부터 재처리가 가능합니다.
- **서버 한 대가 죽어도 메시지는 안전하다**: Kafka는 보통 여러 대의 서버를 클러스터로 묶어서 운영하며, 메시지를 여러 서버에 복제해 둡니다. 따라서 특정 서버 한 대에 장애가 발생하더라도, 복제본을 가진 다른 서버가 즉시 역할을 넘겨받기 때문에 메시지 유실 없이 서비스를 안정적으로 유지할 수 있습니다.

오픈이 : "각 서비스가 메시지 방식으로 처리하다가 중간에 실패하면, 보상 트랜잭션은 어떻게 관리하나요?"

선배 : "그건 흐름을 누가 관리하느냐에 따라 달라요. 지금처럼 각 서비스가 서로 메시지를 발행하고 구독하며, 실패 시에도 직접 보상 메시지를 발행할 수 있어요. 다만 이 방식은 중간에 실패하면 어디까지 됐는지 알기 어려워 보상도 까다로워요. 반대로 전체 흐름을 관리하는 **지휘자**를 두고 지휘자가 메시지 발행과 구독을 관리하면, 어디까지 진행됐는지 알고 있으니 **이미 끝난 단계만 골라 되돌릴** 수 있어요."

4.3 Orchestration Saga - 지휘자가 흐름을 조율하다

하나의 주문을 여러 서비스가 단계별로 처리하다 보면, 중간 한 단계가 실패해도 앞 단계는 이미 데이터에 반영돼 있습니다. 이때 끝난 단계를 취소하는 **보상**으로 데이터를 맞추는 방식을 **Saga 패턴**이라고 합니다.

Saga 패턴에서 전체 흐름을 지휘자가 중앙에서 관리하는 방식을 **Orchestration Saga**라고 합니다. 흐름을 한 곳에 모으면 상태가 한 곳에 있어 추적과 보상이 단순해지고, 각 서비스는 자기 일에만 집중할 수 있습니다.

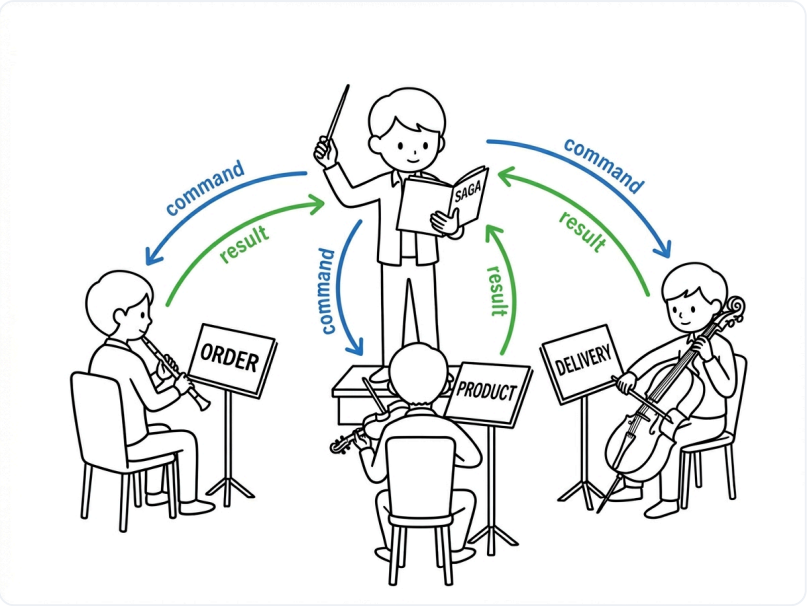


그림 4-6. Orchestration Saga 구조

챕터 2~3에서 구현한 주문 서비스 중심의 보상 트랜잭션 관리도 일종의 Orchestration Saga입니다.

이번 챕터에서는 조율 역할만 전담하는 **별도 orchestrator 서비스**를 두는 구조를 사용합니다. 주문 요청이 들어오면 orchestrator가 재고 차감, 배달 생성, 주문 완료 단계를 순차적으로 지휘합니다.

> **Saga를 구현하는 방식에는 Orchestration 외에 Choreography도 있습니다.** 중앙 지휘자 없이 **각 서비스가 서로 발행하고 구독하며 다음 단계를 이어 가는 방식**으로, 단계가 적을 때는 가볍지만 단계가 늘거나 보상이 복잡해질수록 전체 흐름이 여러 서비스에 흩어져 추적이 어렵습니다. 이 책은 상태를 한 곳에서 추적할 수 있는 Orchestration을 선택합니다.

4.3.1 이번 챕터에서 사용하는 토픽 맵

주문 흐름에서는 총 8개의 토픽을 사용합니다. 토픽은 orchestrator가 서비스에 작업을 지시하는 **명령(Command)** 과, 서비스가 결과를 돌려주는 **이벤트(Event)** 두 종류로 나뉩니다. 구분을 위해 토픽 이름에 `command`가 포함되면 명령, 없으면 이벤트로 정의합니다.

정상 흐름 (6단계)

단계	토픽	발행 → 구독	목적
1	`order-created`	order → orchestrator	새 주문 발생
2	`decrease-product-command`	orchestrator → product	재고 감소 명령
3	`product-decreased`	product → orchestrator	재고 감소 결과
4	`create-delivery-command`	orchestrator → delivery	배달 생성 명령
5	`delivery-created`	delivery → orchestrator	배달 생성 결과
6	`complete-order-command`	orchestrator → order	주문 완료 명령

보상 흐름 (2개)

토픽	발행 → 구독	목적
`cancel-order-command`	orchestrator → order	주문 취소
`increase-product-command`	orchestrator → product	재고 복구

핵심 흐름만 직접 다루므로, 각 토픽의 발행·구독 코드 전체는 깃헙 레포에서 확인합니다.

4.3.2 주문 요청 성공 흐름

이제 비동기 통신으로 주문 요청이 처리되는 전체 흐름을 따라가 보겠습니다.

먼저 주문 요청이 들어오면 주문 서비스는 클라이언트에게 **PENDING** 상태를 응답합니다. 이후 orchestrator가 아래 단계를 차례로 진행해 모두 성공하면 주문 상태가 **COMPLETED**로 바뀝니다.



그림 4-7. 1단계 — 주문 생성 이벤트 발행 → 오케스트레이터 수신

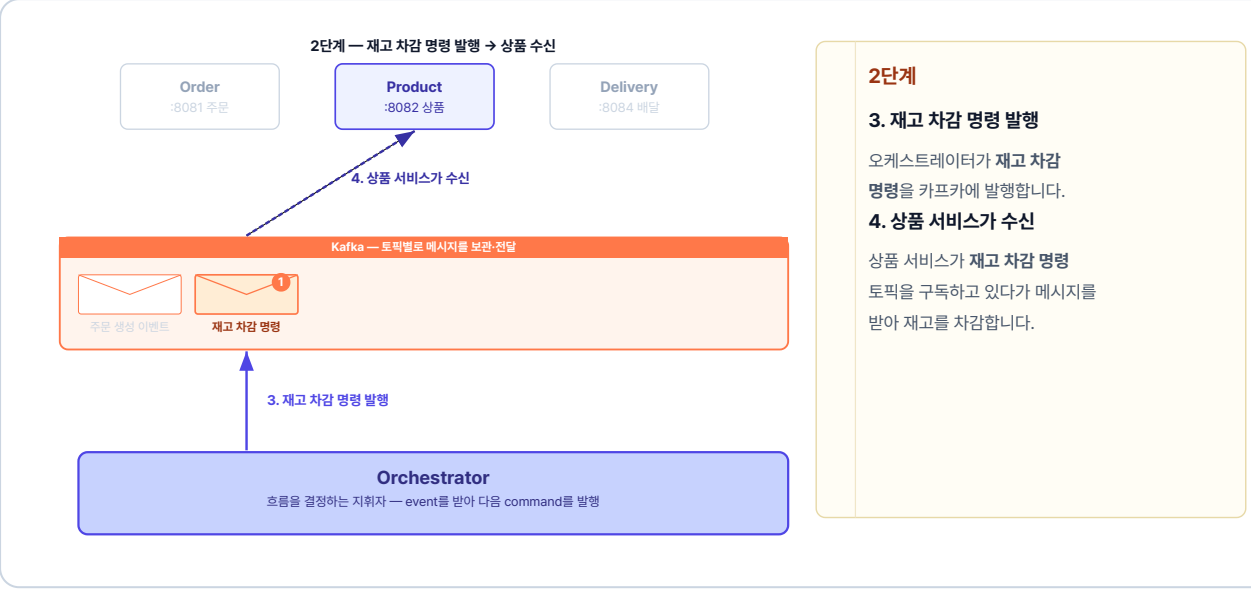


그림 4-8. 2단계 — 재고 차감 명령 발행 → 상품 수신



3단계

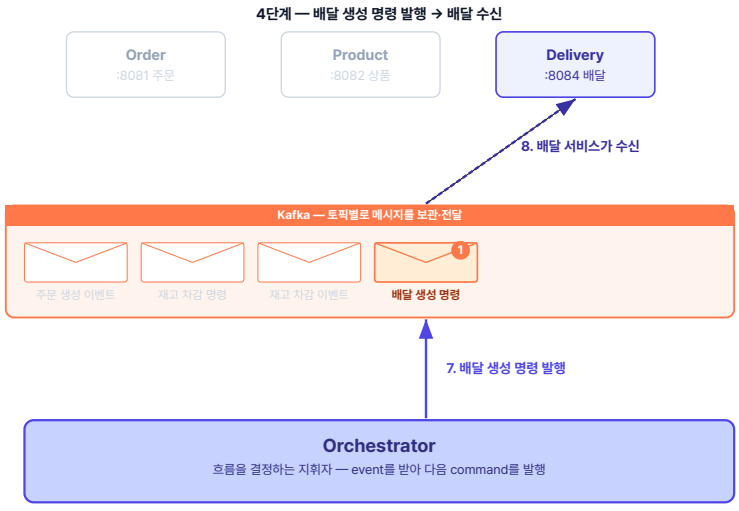
5. 재고 차감 이벤트 발행

상품 서비스가 재고를 줄인 뒤 성공 여부를 **재고 차감 이벤트**로 카프카에 발행합니다.

6. 오케스트레이터가 수신

오케스트레이터가 **재고 차감 이벤트** 토픽을 구독하고 있다가 메시지를 받아 성공을 확인합니다.

그림 4-9. 3단계 — 재고 차감 이벤트 발행(상품) → 오케스트레이터 수신



4단계

7. 배달 생성 명령 발행

오케스트레이터가 **배달 생성 명령**을 카프카에 발행합니다.

8. 배달 서비스가 수신

배달 서비스가 **배달 생성 명령** 토픽을 구독하고 있다가 메시지를 받아 배달을 생성합니다.

그림 4-10. 4단계 — 배달 생성 명령 발행 → 배달 수신

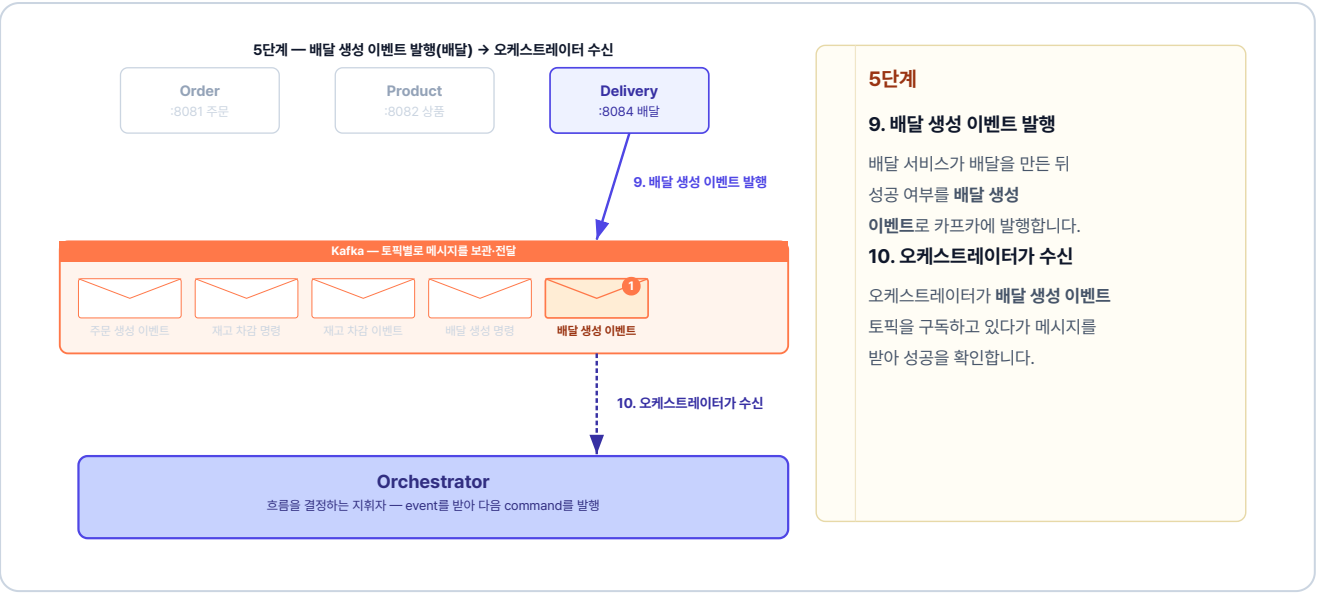


그림 4-11. 5단계 — 배달 생성 이벤트 발행(배달) → 오케스트레이터 수신



그림 4-12. 6단계 — 주문 완료 명령 발행 → 주문 수신

4.3.3 주문 요청 실패 흐름 (보상 트랜잭션)

만약 상품 서비스가 재고 차감에 실패하면 재고 차감 실패 이벤트를 발행합니다. orchestrator는 **이미 처리된 단계만** 역순으로 되돌립니다.

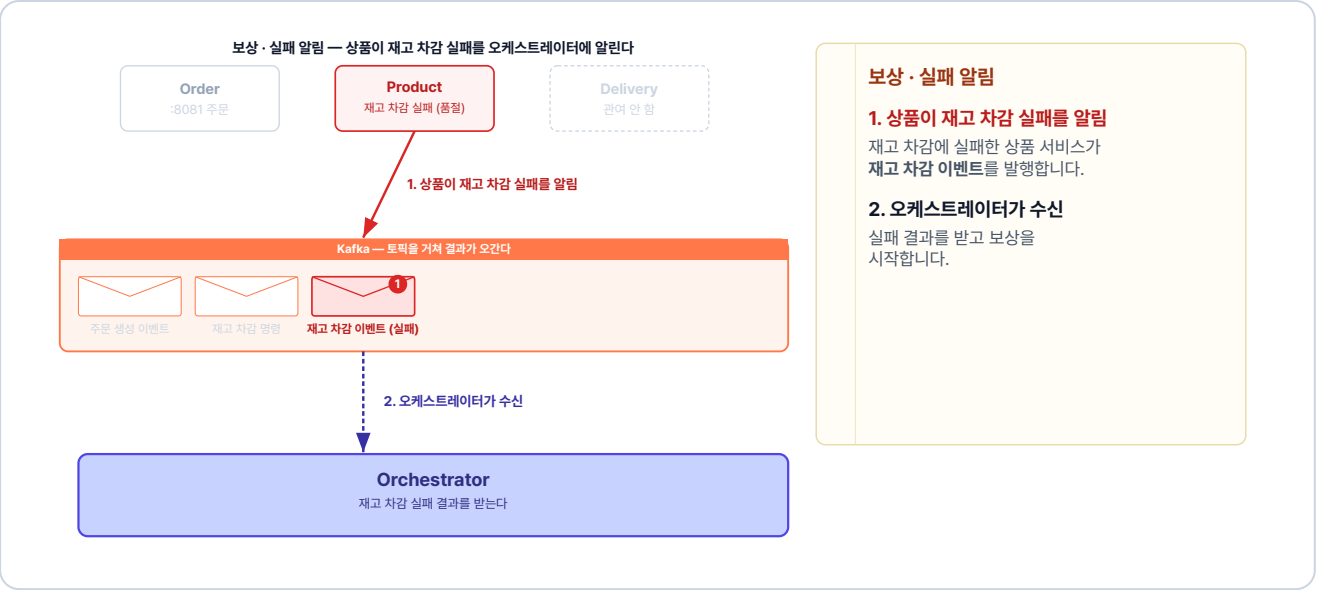


그림 4-13. 보상 · 실패 알림 — 상품이 실패를 알림

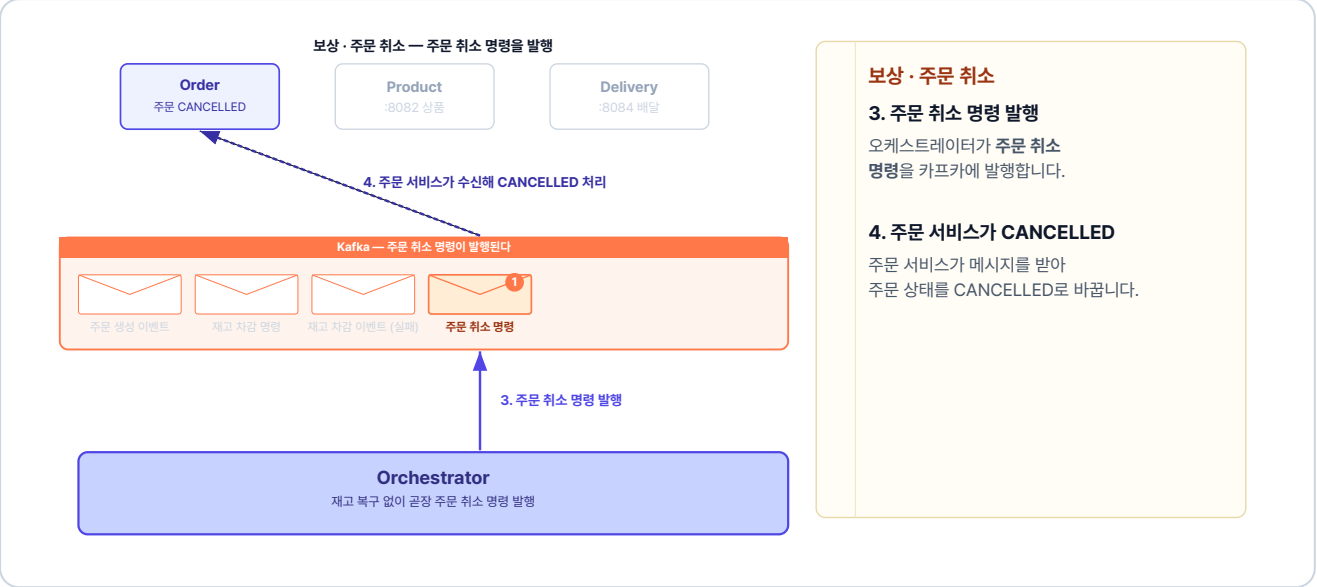


그림 4-14. 보상 · 주문 취소 — 주문 취소 명령 발행

4.4 Kafka로 주고받기 - 발행과 구독

4.4.1 발행 - 토픽에 메시지를 넣는다

프로듀서가 메시지를 발행할 때는 Spring이 제공하는 `KafkaTemplate` 을 사용합니다. **주문 생성 이벤트**를 발행하는 코드는 다음과 같습니다.

주문 서비스의 `adapter/producer/OrderEventProducer.java`를 열고 아래 메서드를 작성합니다.

실습 1 주문 서비스 - adapter/producer/OrderEventProducer.java. 주문 생성 이벤트 발행

```
@Component
@RequiredArgsConstructor
public class OrderEventProducer {
    private final KafkaTemplate<String, Object> kafkaTemplate;

    public void publishOrderCreated(OrderCreatedEvent event) {
        // "order-created" 토픽에 이벤트를 넣는다
        kafkaTemplate.send("order-created", event);
    }
}
```

`kafkaTemplate.send`에 `order-created` 같은 토픽 이름과 보낼 이벤트를 넘기면 해당 토픽으로 메시지가 들어갑니다.

주문 서비스는 주문을 **PENDING** 상태로 저장하고, **주문 생성 이벤트 프로듀서**를 호출합니다.

주문 서비스의 `usecase/OrderService.java`를 열고 `createOrder` 메서드를 작성합니다.

실습 2 주문 서비스 - usecase/OrderService.java. 주문 저장 후 주문 생성 이벤트 발행

```
@Override
@Transactional
public OrderResponse createOrder(int userId, int productId,
        int quantity, Long price, String address) {
    // 1. 주문 생성
    Order createdOrder = orderRepository.save(
            Order.create(userId, productId, quantity, price));
    createdOrder.validateMinAmount();

    // 2. Kafka로 주문 생성 이벤트 발행
    orderEventProducer.publishOrderCreated(
            new OrderCreatedEvent(
                    createdOrder.getId(), userId, productId,
                    quantity, price, address)
    );

    return OrderResponse.from(createdOrder);
}
```

4.4.2 orchestrator - 흐름을 조율하는 코드

이벤트를 받아 **다음 명령을 정하는 일**은 orchestrator가 수행합니다. **주문 생성 이벤트**를 받아 **재고 차감 명령**을 발행하는 코드는 다음과 같습니다.

오케스트레이터 서비스의 `handler/OrderOrchestrator.java`를 열고 아래 메서드를 작성합니다.

실습 3 오케스트레이터 서비스 - handler/OrderOrchestrator.java. 재고 차감 명령 발행

```
@KafkaListener(topics = "order-created", groupId = "orchestrator")
public void orderCreated(OrderCreatedEvent event) {
    // 1. 주문 진행 상태를 메모리에 기록
    states.put(event.orderId(), new WorkflowState(
            event.orderId(), event.address(),
            event.productId(), event.quantity(), event.price()));

    // 2. 다음 단계: 재고 차감 명령 발행
    kafkaTemplate.send(
            "decrease-product-command",
            // 같은 주문은 같은 파티션으로 (순서 보장)
            String.valueOf(event.orderId()),
            new DecreaseProductCommand(event.orderId(),
                    event.productId(), event.quantity(), event.price()));
}
```

`@KafkaListener`의 `topics`는 구독할 토픽 이름, `groupId`는 컨슈머 그룹 이름입니다. 그리고 `WorkflowState`는 **주문의 진행 정보를 메모리에 들고 있는 객체**로, 결과 이벤트가 돌아오면 orchestrator는 이 기록을 보고 다음 명령을 정합니다.

4.4.3 구독 - 토픽의 메시지를 받는다

앞에서 orchestrator가 발행한 **재고 차감 명령**을 이번에는 상품 서비스가 받습니다. 받은 명령으로 재고를 줄이고, 그 결과를 **재고 차감 이벤트**로 발행합니다.

상품 서비스의 `adapter/consumer/ProductCommandConsumer.java`를 열고 아래 메서드를 작성합니다.

실습 4 상품 서비스 - adapter/consumer/ProductCommandConsumer.java. 재고 차감 명령 구독

```
@KafkaListener(topics = "decrease-product-command", groupId = "product-service")
public void decreaseProductCommand(DecreaseProductCommand command) {
    boolean isSuccess = false;
```

```
    // 1. 재고 차감 (성공하면 isSuccess = true)
    try {
        productService.decreaseQuantity(
                command.productId(), command.quantity(), command.price());
        isSuccess = true;
    } catch (Exception e) {
        // 재고 부족 등 실패는 isSuccess = false로 그대로 보고
    }

    // 2. 처리 결과를 '재고 차감 이벤트'로 발행
    productEventProducer.publishProductDecreased(
            new ProductDecreasedEvent(
                    command.orderId(), command.productId(),
                    command.quantity(), isSuccess));
}
```

상품 서비스는 명령을 받아 재고를 줄인 뒤, 성공이든 실패든 그 결과를 이벤트에 담아 돌려줍니다.

각 서비스 코드는 모두 같은 발행·구독 패턴이고, 토픽 이름만 다릅니다. 그래서 코드를 일일이 보지 않아도, 각 서비스가 무슨 토픽을 구독해 어떻게 처리하고 무엇을 발행하는지만 알면 전체 흐름이 보입니다.

각 서비스의 전체 코드는 GitHub에서 확인하세요.

이제 Kubernetes에 Kafka와 orchestrator를 추가하고 전체 시스템을 배포합니다.

4.5 Kubernetes - Kafka와 orchestrator 배포

Kafka를 도입하면서 추가되는 건 **Kafka 서버**와 **orchestrator 서버**입니다.

Kubernetes 리소스	역할
`kafka-deploy.yml`	Kafka를 실행하는 Pod를 정의하고 원하는 상태로 유지합니다.
`kafka-service.yml`	Kafka Pod에 고정 주소 `kafka-service:9092`를 부여해, 주문·상품 등 각 서비스가 이 주소로 Kafka에 연결합니다.
`orchestrator-deploy.yml`	orchestrator를 실행하는 Pod를 정의합니다.
`orchestrator-configmap.yml`	orchestrator에 Kafka 주소 같은 설정값을 환경변수로 주입합니다.

모든 서비스는 Kafka 서버의 주소 `kafka-service:9092`로 접근합니다. 각 서비스는 이 주소를 ConfigMap에 넣어 둡니다.

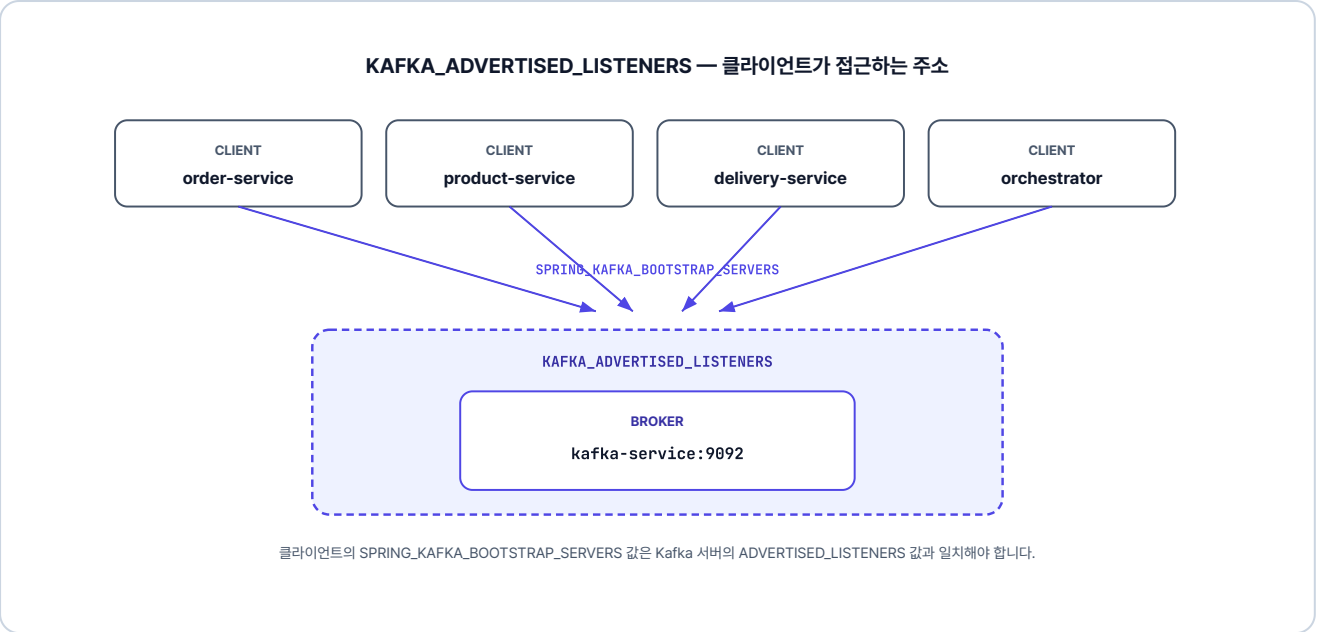


그림 4-15. 클라이언트가 kafka-service:9092로 접근

`kafka-deploy.yml`의 전체 환경변수는 GitHub에서 확인하세요. 각 변수의 역할은 주석으로 달려 있습니다.

KRaft 모드 알아두기

Kafka 서버는 크게 두 가지 역할을 합니다. **메시지를 받아 전달하는 브로커**, 그리고 **클러스터와 토픽 설정을 관리하는 컨트롤러**입니다.

과거에는 이 컨트롤러 역할을 **ZooKeeper**라는 외부 서비스에 따로 맡겨야 했습니다. 하지만 **KRaft(Kafka Raft)** 모드에서는 Kafka가 관리 역할까지 직접 도맡습니다. 덕분에 복잡한 ZooKeeper 연동 없이 Kafka 컨테이너 하나만으로 두 가지 역할을 모두 처리합니다.

실제 운영 환경에서는 데이터 유실을 막기 위해 여러 대의 브로커 노드를 구성해 안정성을 높입니다. 다만 이 책에서는 실습의 편의를 위해 하나의 컨테이너에 브로커와 컨트롤러를 함께 구성해 진행합니다.

4.6 실행 및 결과 확인

4.6.1 이미지 빌드

Minikube 내부에 이미지를 빌드합니다. 챕터 3 대비 orchestrator 서비스가 새로 추가됩니다.

터미널 이미지 빌드

```
minikube image build -t metacoding/db:2 ./db
minikube image build -t metacoding/gateway:2 ./gateway
minikube image build -t metacoding/order:2 ./order
minikube image build -t metacoding/product:2 ./product
minikube image build -t metacoding/user:2 ./user
minikube image build -t metacoding/delivery:2 ./delivery
minikube image build -t metacoding/orchestrator:2 ./orchestrator
```

4.6.2 배포 순서

Kafka가 준비되기 전에 서비스가 시작되면 연결 오류가 발생합니다. Kafka를 먼저 배포하고 준비된 것을 확인한 다음 나머지를 배포합니다.

터미널 배포 순서 (Kafka 우선)

```
# 1. 네임스페이스 생성
kubectl create namespace metacoding

# 2. Kafka 먼저 배포
kubectl apply -f k8s/kafka

# 3. Kafka가 준비될 때까지 대기
kubectl wait --for=condition=ready pod -l app=kafka -n metacoding --timeout=120s

# 4. 나머지 서비스 배포
kubectl apply -f k8s/db
kubectl apply -f k8s/order
kubectl apply -f k8s/product
kubectl apply -f k8s/user
kubectl apply -f k8s/delivery
kubectl apply -f k8s/gateway
kubectl apply -f k8s/orchestrator

# 5. Ingress 활성화 (최초 1회)
minikube addons enable ingress
```

kubectl apply · Kafka + 서비스 배포

```
namespace/metacoding created

[1] Kafka 우선 배포
deployment.apps/kafka-deploy created
service/kafka-service created
pod/kafka-xxx condition met (28s)

[2] 나머지 서비스 배포
  db · order · product · user · delivery · gateway · orchestrator …

8개 Deployment + 7개 Service 배포 완료
```

그림 4-16. Kafka 및 서비스 배포 실행

모든 Pod가 Running 상태인지 확인합니다.

터미널 **Pod 상태 확인**

```
kubectl get pods -n metacoding
```

kubectl get pods -n metacoding

```
NAME                                READY   STATUS    AGE
kafka-deploy-7d4c8b9f5-2xk9p        1/1     Running   2m
db-deploy-6f9b7c4d8-m4t2q           1/1     Running   90s
gateway-deploy-5c8d6f7b9-h7w3r      1/1     Running   88s
order-deploy-8b7f6c9d4-q2k8m        1/1     Running   85s
product-deploy-7c9d8b6f5-x4r2t      1/1     Running   83s
user-deploy-6d8c7b9f4-p3m9k         1/1     Running   80s
delivery-deploy-9f7c8b6d5-t6w2x     1/1     Running   78s
orchestrator-deploy-8c6f9b7d4-k9m4q 1/1     Running   75s

8개 Pod Running (Kafka + orchestrator 추가)
```

그림 4-17. Pod 상태 확인 (kubectl get pods)

4.6.3 서비스 접근

Ingress를 통해 외부에서 접속하기 위해 `minikube tunnel`을 실행합니다.

터미널 외부 접근 터널

```
minikube tunnel
```

터널이 실행되면 `http://127.0.0.1:80`로 gateway-service에 접속할 수 있습니다.

4.6.4 비동기 흐름 테스트

이제 AirPods (productId=3) 2개를 주문하는 API 요청을 보내 보겠습니다.

Hoppscotch 주문 생성

```
POST http://127.0.0.1:80/api/orders

{
  "productId": 3,
  "quantity": 2,
  "price": 300000,
  "address": "Addr 4"
}
```



그림 4-18. 주문 생성 응답 (PENDING 상태)

챕터 3과 다르게 즉시 `PENDING` 상태로 반환됩니다. 잠시 후 주문 상태를 다시 조회하면, Kafka 이벤트가 처리되어 상태가 `COMPLETED`로 바뀐 것을 확인할 수 있습니다.

Hoppscotch 주문 조회

```
GET http://127.0.0.1:80/api/orders/4
```



그림 4-19. 주문 완료 확인 (COMPLETED 상태)

4.6.5 보상 트랜잭션 확인 - 품절 상품 주문

동기 방식에서는 주문 서비스가 트랜잭션을 관리했기 때문에 주문이 실패하면 주문 데이터가 **자동 롤백**되었습니다. 반면 비동기 방식에서는 주문이 **PENDING으로 먼저 저장**됩니다. 그래서 재고 감소가 실패하면 **보상 트랜잭션**에 의해 주문 상태가 `CANCELLED`로 변경됩니다.

iPhone 15(productId=2, 재고 0)로 확인해 보겠습니다.

Hoppscotch 주문 생성 (품절 상품)

```
POST http://127.0.0.1:80/api/orders

{
  "productId": 2,
  "quantity": 1,
  "price": 1300000,
  "address": "Addr 5"
}
```



그림 4-20. 품절 상품 주문 요청

잠시 후 상태를 확인하면 `CANCELLED`가 됩니다.

Hoppscotch 주문 조회

```
GET http://127.0.0.1:80/api/orders/5
```



그림 4-21. 주문 취소 확인 (CANCELLED 상태)

테스트가 끝났으면 이번 챕터에서 실행한 리소스를 정리합니다.

터미널 **리소스 정리**

```
kubectl delete namespace metacoding
```

오픈이 : "이제 직접 호출하지 않고 메시지로 주고받으니까, 다른 서비스가 일시적으로 멈춰도 주문은 계속 받을 수 있겠네요."

선배 : "맞아요. 서비스 간의 결합이 느슨해진 덕분이죠. 게다가 주문이 한꺼번에 몰려도 Kafka가 중간에서 메시지를 받아 두니, 받는 쪽은 감당할 수 있는 속도로 처리하면 돼요. 한 서비스에 장애가 나거나 부하가 걸려도 시스템 전체가 무너지지 않아요."

4.7 더 알아보기 - Outbox 패턴

오픈이 : "그런데 한 가지 걸리는 게 있어요. 주문을 저장하고 Kafka로 발행하는데, 발행이 실패하면 어떻게 되죠? 주문은 DB에 남아 있는데 이벤트만 사라지는 거 아닌가요?"

선배 : "정확히 봤어요. 그냥 넘기면 안 되는 부분이에요. 그걸 해결하는 방법으로 **Outbox 패턴**을 사용해요."

MSA에서는 보통 DB 변경과 메시지 발행을 함께 처리합니다. **문제는 두 작업이 서로 다른 시스템에서 일어나 하나의 트랜잭션으로 묶을 수 없다는 점입니다.** 그래서 주문 생성은 성공했는데 네트워크 오류나 서버 다운으로 발행만 실패하면, 오케스트레이터가 주문 생성을 알지 못해 상품·배달 처리가 시작되지 않고, 데이터가 어긋납니다.

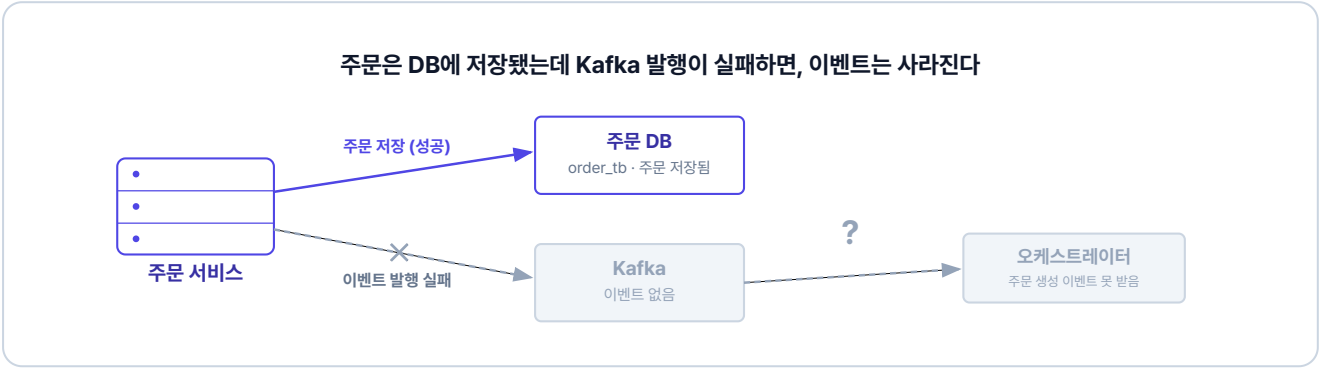


그림 4-22. 직접 발행의 한계 - DB 저장과 Kafka 발행이 별개라, 발행이 실패하면 이벤트가 유실됩니다

이 문제를 해결하는 것이 **Outbox 패턴**입니다.

먼저 주문 테이블과 같은 DB에 outbox 테이블을 하나 더 만듭니다. 그리고 주문 데이터를 저장할 때, 발행할 메시지도 outbox 테이블에 함께 저장합니다. 둘 다 같은 DB에 있으니 하나의 트랜잭션으로 묶을 수 있습니다. 주문과 메시지가 같이 커밋되거나 같이 롤백되므로, 주문만 저장되고 이벤트가 사라지는 일이 없어집니다.

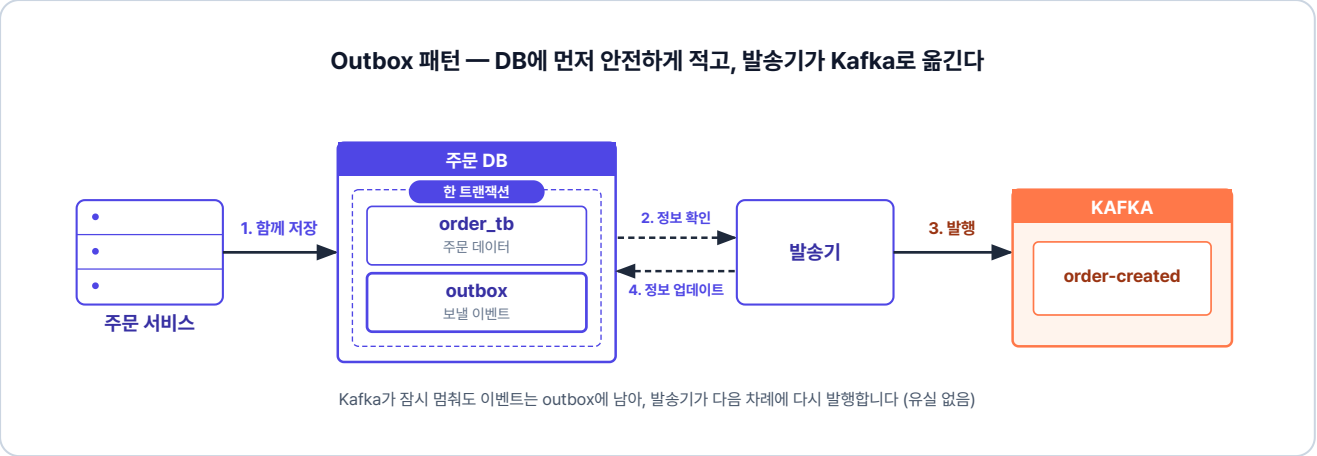


그림 4-23. Outbox 패턴 - 주문과 이벤트를 한 트랜잭션으로 저장한 뒤, 발송기가 outbox를 읽어 Kafka로 옮깁니다

그다음 **발송기(Relay)** 가 outbox 테이블을 주기적으로 확인합니다. 아직 발행하지 않은 메시지가 있으면 Kafka로 발행한 뒤, outbox 테이블에서 그 메시지의 상태를 보냄으로 업데이트합니다. 덕분에 발행이 실패해도 메시지가 outbox에 남아 있어, 다시 발행할 수 있습니다.

Outbox만으로 충분할까?

Outbox 패턴은 이벤트 유실을 막아주지만, 중복 발송까지 막지는 못합니다. 발송기가 이벤트를 보낸 직후 완료 처리를 하기 전에 멈추면, 다음 차례에 같은 이벤트를 또 보내기 때문입니다. 그래서 이벤트를 받는 쪽은 똑같은 요청을 여러 번 받아도 한 번만 처리한 것과 같은 결과를 내도록 방어해야 합니다. 이러한 성질을 **멱등성**이라고 합니다. 실무에서는 보통 이미 처리한 이벤트 ID를 기록해 두고, 같은 ID가 다시 들어오면 건너뛰는 방식으로 구현합니다.

이렇게 실시간 유실과 중복을 막더라도, 예상치 못한 시스템 오류로 데이터가 어긋날 수 있습니다. 따라서 규모가 큰 서비스에서는 주기적으로 누락되거나 어긋난 부분들을 찾아 배치 작업으로 보정합니다.

이 책에서는 패턴의 개념까지만 소개합니다. 구현은 다루지 않지만, 이벤트로 주고받는 시스템에서는 거의 빠지지 않는 장치입니다.

한편 현재 구조에서는 클라이언트가 처음에 **주문 대기** 상태만 응답받을 뿐, 이후 실제 처리가 완료되어도 그 결과를 따로 알 수 없습니다. 다음 챕터에서는 **웹소켓(WebSocket)** 을 도입해 이 문제를 해결하고, 주문 처리가 끝나는 즉시 클라이언트에게 실시간 알림을 보내는 방법을 알아보겠습니다.

이것만은 기억하자

- → 서비스끼리 REST로 직접 호출하는 대신 **Kafka**로 메시지를 주고받습니다. 이 **비동기 방식**으로 서비스 간 결합이 느슨해집니다.
- → **orchestrator**가 **명령**으로 각 서비스를 조율하고, 각 서비스는 **이벤트**로 결과를 알립니다.
- → **Saga 패턴**으로 분산 트랜잭션을 단계별로 처리하고, 실패한 단계는 **보상 트랜잭션**으로 역순으로 되돌립니다.
- → **Outbox 패턴**으로 주문과 이벤트를 한 트랜잭션으로 함께 저장해, 발행 실패로 인한 **이벤트 유실**을 막습니다.